

Planejamento de testes - ServeRest

Planejamento de testes - ServeRest

1. Visão Geral da Sessão

- **ID:** TESTE-001
- **Aplicação:** ServeRest
- **Squad:** Breaking Bugs
- **Testador:** Julliely de Sousa Silva
- **Data:** 11/08/2025
- **Início:** 11/08/2025
- **Fim:** 18/08/2025

APRESENTAÇÃO

O presente planejamento de testes foi desenvolvido com base no estudo e aplicação de técnicas relacionadas à qualidade de software, focando em **APIs REST**. O objetivo central é garantir que a API **ServeRest** (<https://compassuol.serverest.dev/>) funcione corretamente, atendendo às **regras de negócio** estabelecidas nas User Stories fornecidas, bem como às especificações do Swagger.

Este documento visa organizar e detalhar todas as etapas do processo de testes, incluindo a análise das funcionalidades, definição de cenários de teste, priorização, mapeamento de riscos e cobertura de testes, contribuindo para a entrega de um produto confiável, seguro e com alto padrão de qualidade.

O planejamento contempla tanto a parte de **testes manuais** quanto a **possibilidade de automação**, garantindo que os testes possam ser repetidos e escaláveis ao longo do ciclo de vida da aplicação.

OBJETIVO

O objetivo deste plano de testes é:

- Garantir a **qualidade funcional** da API ServeRest, verificando se as funcionalidades de cadastro de usuários, autenticação e gerenciamento de produtos atendem às regras de negócio especificadas.

- Identificar e registrar **inconsistências ou erros** na aplicação, tanto em cenários positivos quanto alternativos.
- **Planejar, organizar e priorizar** a execução dos testes, fornecendo evidências claras e rastreabilidade entre requisitos, testes e resultados.
- Subsidiar a **automação de testes**, fornecendo scripts e validações que possam ser executadas de forma repetitiva, garantindo eficiência e confiabilidade contínua.

ESCOPO

O escopo deste planejamento de testes inclui:

1. APIs e funcionalidades a serem testadas:

- Cadastro de usuários (CRUD) conforme **US 001**.
- Autenticação de usuários e geração de token Bearer conforme **US 002**.
- Cadastro, atualização, listagem e exclusão de produtos conforme **US 003**.

2. Tipos de testes planejados:

- Testes funcionais das APIs (verificação dos endpoints, verbos HTTP e regras de negócio).
- Testes de validação de dados, incluindo regras de e-mail, senha e restrições de duplicidade.
- Testes de segurança relacionados à autenticação e controle de acesso.

3. Limitações do escopo:

- Testes relacionados a APIs externas ou dependentes (ex.: carrinhos de compras) serão considerados somente nas interações críticas.
- Cenários complexos de performance ou carga não estão contemplados neste planejamento inicial.

ANÁLISE

A análise da API **ServeRest** tem como objetivo fornecer uma **visão estratégica** do produto, servindo como ponte entre as funcionalidades da aplicação e os riscos que orientam a priorização dos testes. Ela

considera tanto os **aspectos de produto** quanto as **regras de negócio** definidas nas User Stories (US 001, US 002 e US 003) e no Swagger da API.

Aspectos estratégicos identificados:

1. Visão do Produto

- A API ServeRest possibilita que vendedores se cadastrem, autentiquem e gerenciem produtos dentro de um marketplace.
- Funcionalidades críticas incluem cadastro de usuários, autenticação segura e gerenciamento de produtos (CRUD).
- O produto deve suportar corretamente as operações típicas de um marketplace, garantindo integridade e consistência de dados.

2. Regras de Negócio Relevantes

- Campos obrigatórios e validações de entrada (e-mails válidos, restrições de provedores, tamanho de senha).
- Proteção contra duplicidade (usuários e produtos).
- Dependências entre APIs (ex.: produtos não podem ser excluídos se estiverem em carrinhos).
- Geração de tokens de autenticação e restrições de acesso para usuários não autenticados.

3. Riscos Estratégicos que Orientam a Prioridade dos Testes

- **Segurança e autenticação:** falhas podem permitir acesso não autorizado.
- **Integridade de dados:** duplicidade ou exclusão indevida de registros impacta diretamente a operação do marketplace.
- **Confiabilidade das APIs:** endpoints indisponíveis ou mal implementados podem impedir operações críticas de venda.
- **Conformidade com regras de negócio:** ausência de validação de e-mails ou senhas impacta a experiência do usuário e integridade do sistema.

4. Conclusão Estratégica da Análise

- A análise permitiu identificar **prioridades para os testes**, concentrando esforços em áreas críticas para a operação do marketplace: autenticação, cadastro de usuários e gerenciamento de produtos.

- Essa visão estratégica sustenta a definição de **cenários de teste**, **matriz de risco** e **cobertura de testes**, garantindo que os testes executados agreguem valor real ao produto e reduzam potenciais falhas em produção.

TÉCNICAS APLICADAS

Para garantir a efetividade dos testes, foram aplicadas as seguintes **técnicas de planejamento de testes**:

1. Partição de Equivalência

- Para validação de campos de cadastro (ex.: e-mails válidos e inválidos, tamanho da senha).
- Reduz o número de casos de teste mantendo cobertura de cenários relevantes.

2. Análise de Valor Limite

- Testes nas fronteiras de tamanho de senha, quantidade de produtos e IDs de usuários.
- Garante que o sistema lida corretamente com valores mínimos, máximos e próximos às fronteiras.

3. Técnica de Cenários Positivos e Negativos

- Positivos: operações de CRUD bem-sucedidas.
- Negativos: tentativas de duplicidade, autenticação inválida, exclusão de recursos dependentes.

4. Mapeamento de Dependências

- Avaliação de interações entre usuários, produtos e carrinhos.
- Identificação de possíveis impactos em cascata devido a operações em uma API dependente.

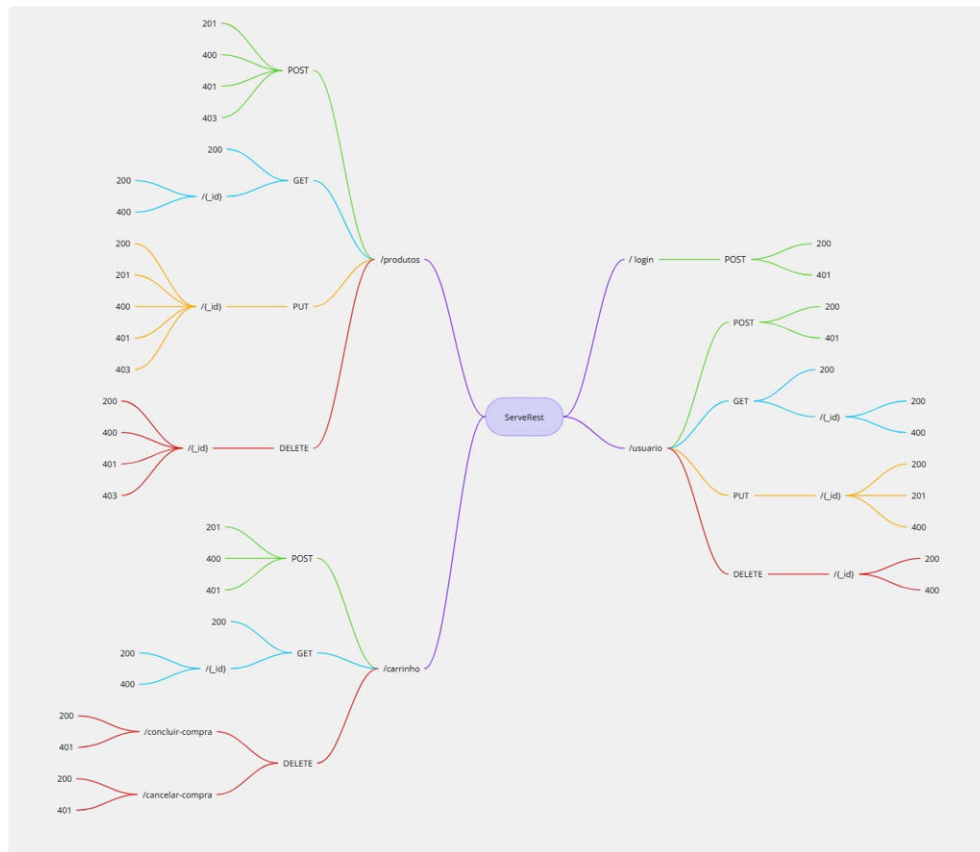
5. Rastreabilidade de Requisitos

- Cada teste planejado está vinculado a uma **User Story** e ao **endpoint Swagger** correspondente.
- Facilita a documentação de evidências e abertura de issues.

6. Planejamento para Automação

- Identificação de testes candidatos à automação via Postman (aba **Tests**) e geração de scripts de validação.
- Foco em testes repetitivos, críticos e que envolvam validação de regras de negócio.

MAPA MENTAL DA APLICAÇÃO



CENÁRIOS DE TESTES PLANEJADOS

1. Usuários (US 001)

Cenários Positivos:

1. Cadastro de novo usuário válido

- Verifica se um usuário pode ser cadastrado com nome, e-mail válido, senha dentro do limite e flag de administrador.

2. Atualização de usuário existente

- Verifica se as alterações nos dados de um usuário existente são aplicadas corretamente.

3. Listagem de usuários cadastrados

- Confirma se a API retorna corretamente todos os usuários existentes.

4. Exclusão de usuário existente

- Verifica se um usuário pode ser removido da base de dados sem afetar integridade do sistema.

Cenários Alternativos / Negativos:

5. Cadastro de usuário com e-mail duplicado

- Testa se a API impede criação de usuário com e-mail já cadastrado.

6. Cadastro de usuário com e-mail inválido (gmail ou hotmail)

- Verifica se o sistema bloqueia provedores não permitidos.

7. Cadastro de usuário com senha fora do limite (menos de 5 ou mais de 10 caracteres)

- Avalia se o sistema aplica corretamente a validação de senha.

8. Atualização de usuário inexistente via PUT

- Verifica se um novo usuário é criado automaticamente ao tentar atualizar um ID inexistente.

2. Login (US 002)

Cenários Positivos:

1. Autenticação de usuário válido

- Usuário cadastrado realiza login corretamente e recebe token Bearer válido.

Cenários Alternativos / Negativos:

2. Autenticação de usuário não cadastrado

- Verifica se a API retorna erro 401 (Unauthorized).

3. Autenticação com senha incorreta

- Garante que o sistema não autentica e retorna status 401.

4. Validade do token Bearer

- Confirma que o token gerado expira corretamente após 10 minutos.

3. Produtos (US 003)

Cenários Positivos:

1. Cadastro de novo produto válido

- Usuário autenticado cadastra produto com dados válidos.

2. Atualização de produto existente

- Verifica se produto existente é atualizado corretamente.

3. Listagem de produtos

- Confirma que todos os produtos cadastrados são retornados corretamente.

4. **Exclusão de produto sem dependência em carrinho**

- Produto é removido sem afetar outras APIs.

Cenários Alternativos / Negativos:

5. **Cadastro de produto com nome duplicado**

- Testa se o sistema bloqueia produtos com nomes já existentes.

6. **Exclusão de produto presente em carrinho**

- Avalia se a API impede exclusão de produtos vinculados a carrinhos.

7. **Atualização de produto inexistente via PUT**

- Verifica se um novo produto é criado automaticamente ao tentar atualizar um ID inexistente.

8. **Ações de produtos por usuário não autenticado**

- Testa se a API retorna erro 401 para usuários não logados.

Esses cenários podem ser **priorizados** de acordo com **impacto no negócio e risco**:

- Prioridade alta: autenticação, cadastro de usuários, integridade de produtos.
- Prioridade média: atualizações e listagens de usuários e produtos.
- Prioridade baixa: cenários alternativos menos críticos, como tentativas de cadastro bloqueadas por e-mail inválido.

MATRIZ DE RISCO

[Elaborar uma matriz de risco e definir em questão de probabilidade e impacto, sua posição em uma tabela.

Probabilidade \ Impacto	Sem impacto	Leve	Médio	Grave	Gravíssimo
Quase certo	Cadastro com e-mail		Falha em validação de dados	Usuário não autenticado	Perda de dados de usuário/

	inválid o (bloqu eio correto)		obrigatóri os	do acessa rota de produtos	produto crítico
Alta			Falha em autentica ção de login	Exclusão de produto em carrinho	Falha na criação/a tualizaçã o de usuário crítico
Média		Cadastro de usuário com e-mail duplicado	Atualizaç ão de produto inexistent e via PUT	Falha em geração de token	
Baixa		Listagem de usuários/p rodutos com dados inconsiste ntes	Atualizaç ão de usuário existente		
Raro				Falha em endpoint s pouco usados	Falha geral de API durante pico de uso

Legenda:

Cinza - Baixo risco

Verde - Risco moderado

Amarelo - Risco elevado

Vermelho - Risco extremo

COBERTURA DE TESTES

Foram utilizadas dois métodos de análise de cobertura de testes, eles são:

1. **Status Code Coverage (Cobertura de Código de Status):** Este método de cobertura verifica se o conjunto de testes executa todos os códigos de resposta HTTP possíveis para cada endpoint. Seu conjunto de testes aborda cenários de sucesso (`200 OK` , `201 Created`), falhas do cliente (`400 Bad Request` , `401 Unauthorized` , `403 Forbidden`) e casos de "não encontrado" (`404 Not Found`). Essa abordagem garante que a API responde corretamente a diferentes tipos de requisições, sejam elas bem-sucedidas ou mal-sucedidas.
2. **Parameter Value Coverage (Cobertura de Valor de Parâmetro):** Este método foca na validação dos dados de entrada (input) que a API recebe, incluindo a verificação de valores válidos, inválidos e de limite. Nos seus testes, foram utilizados:
 - **Valores Válidos:** Dados que estão em conformidade com o formato e as restrições esperadas (ex: login com credenciais corretas).
 - **Valores Inválidos:** Dados que não seguem as regras de validação da API (ex: e-mail ou senha incorretos, campos vazios, ID inexistente).
 - **Valores de Limite (Boundary):** Dados que testam os limites da validação (ex: senhas com 5 e 10 caracteres, que são os limites aceitáveis).

A tabela a seguir detalha a aplicação desses métodos para cada endpoint testado na sua coleção.

Endpoint	Método	Status Codes Testados	Parâmetros Testados	Descrição da Cobertura
POST Login	POST	200, 400, 401	Válidos: E-mail e senha corretos. Inválidos: E-mail inválido, senha incorreta, campos vazios.	Alta cobertura, testando autenticação com sucesso e diversas falhas de autenticação.
POST Criar usuário	POST	201, 400	Válidos: Nome, e-mail e senha corretos. Inválidos: E-mail duplicado, campos vazios, e-mails de domínios específicos. Limites: Senhas fora dos limites de 5 a 10 caracteres.	Cobertura abrangente para criação de usuários, incluindo valores válidos, inválidos e limites de senha.
GET Listar usuários	GET	200	Válidos: Listar todos os usuários, filtrar por nome e e-	Boa cobertura para listagem de usuários

			mail válidos. Inválidos: Filtro por nome inexistente, filtro com parâmetro inválido.	com e sem filtros.
GET Buscar por ID	GET	200, 400	Válidos: ID existente. Inválidos: ID inexistente, ID com tamanho inválido.	Cobertura para busca de usuários por ID, testando cenários de sucesso e falha.
PUT Editar usuário	PUT	200, 201	Válidos: Atualização completa e parcial de usuário existente. Inválidos: ID inexistente (que cria um novo usuário). Limites: Senhas com 5 e 10 caracteres.	Boa cobertura para atualizações, testando dados válidos, limites e o comportamento com ID inexistente.
DELETE usuário	DELETE	200, 400	Válidos: ID existente. Inválidos: ID inexistente, ID com	Cobertura para exclusão de usuário, testando

			caracteres inválidos.	cenários de sucesso e falha.
GET Produtos	GET	200, 400, 401, 403	Válidos: Listar produtos com token válido, filtrar por nome válido. Inválidos: Sem autenticação, filtros inválidos.	Boa cobertura, incluindo cenários de autenticação e filtros.
GET Produtos por ID	GET	200, 400	Válidos: ID de produto existente. Inválidos: ID de produto inválido ou inexistente.	Cobertura para busca de produtos por ID, testando cenários de sucesso e falha.
POST Produtos	POST	201, 400	Válidos: Dados de produto válidos. Inválidos: Nome muito longo, preço negativo, quantidade negativa.	Cobertura abrangente para criação de produtos, incluindo validação de dados de entrada e seus limites.
DELETE Produtos por	DELETE	200, 401, 403	Válidos: Exclusão	Cobertura para

ID			com token de administrador válido. Inválidos: Sem autenticação, usuário sem permissão de administrador.	exclusão de produtos, testando cenários de sucesso e falhas de autenticação/autorização.
PUT Produtos por ID	PUT	200, 201, 400, 401	Válidos: Atualização de produto com dados válidos. Inválidos: ID inexistente, sem autenticação, dados inválidos no corpo.	Alta cobertura para atualizações, testando autenticação, dados inválidos e o comportamento com ID inexistente.

TESTES CANDIDATOS A POSSÍVEL AUTOMAÇÃO

os testes mais adequados para automação, baseando-se em repetitividade, criticidade e possibilidade de reaproveitamento:

- **Login (US 002):**

Autenticação válida e inválida → repetidos em quase todos os fluxos, excelente para automação.

- **Cadastro de usuários (US 001):**

- Usuário válido
- E-mail duplicado

- Campos obrigatórios vazios
- Senha inválida (curta ou longa)

- **Produtos (US 003):**

- Cadastro de produto válido
- Exclusão de produto sem dependência
- Tentativa de exclusão com carrinho vinculado
- Atualização de produto existente

Esses cenários podem ser automatizados.